# Homework #1

CSE 546: Machine Learning
Tyler Shakibai

Collaborators: Jake Murray, Peter Sushko, Michele Martino, Mohammad Al-Jarrah

## Short Answer and "True or False" Conceptual questions

A1. The answers to these questions should be answerable without referring to external materials. Briefly justify your answers with a few words.

a. *[2 points]* In your own words, describe what bias and variance are. What is the bias-variance tradeoff?

Bias is a measure of the expected difference (error) between a model's predicted value and ground truth (true value). Variance is a measure of a model's sensitivity to variations in the data, or how much the model deviates depending on which data it fits. The bias-variance tradeoff is the inverse relationship between bias and variance. Models with lower bias tend to have higher variance and vice versa.

b. *[2 points]* What **typically** happens to bias and variance when the model complexity increases/decreases?

As the model complexity increases, bias decreases and variance increases. As the model complexity decreases, bias increases and variance decreases.

c. *[2 points]* True or False: Suppose you're given a fixed learning algorithm. If you collect more training data from the same distribution, the variance of your predictor increases.

False. In a fixed learning algorithm, if we collect more training data from the same distribution, we expect the predictor's variance to decrease and be less prone to overfitting.

d. *[2 points]* Suppose that we are given train, validation, and test sets. Which of these sets should be used for hyperparameter tuning? Explain your choice and detail a procedure for hyperparameter tuning.

We should tune the hyperparameters on the validation set. We use the train set to optimize our model's parameter values. We then use the validation set to optimize our model's hyperparameters, which determine the model's architecture. We should never use the test set to tune the model's parameters or hyperparameters, this can lead to overfitting.

e. *[1 point]* True or False: The training error of a function on the training set provides an overestimate of the true error of that function.

False. The training error of a function on the training set underestimates the true error of that function. In other words, the training error is bounded by the true error for a given function.

# Maximum Likelihood Estimation (MLE)

A2. You're the Reign FC manager, and the team is five games into its 2021 season. The numbers of goals scored by the team in each game so far are given below:

$$[2, 4, 6, 0, 1].$$

Let's call these scores $x_1, \ldots, x_5$. Based on your (assumed iid) data, you'd like to build a model to understand how many goals the Reign are likely to score in their next game. You decide to model the number of goals scored per game using a *Poisson distribution*. Recall that the Poisson distribution with parameter $\lambda$ assigns every non-negative integer $x = 0, 1, 2, \ldots$ a probability given by

$$\text{Poi}(x|\lambda) = e^{-\lambda}\frac{\lambda^x}{x!}.$$

a. *[5 points]* Derive an expression for the maximum-likelihood estimate of the parameter $\lambda$ governing the Poisson distribution in terms of goal counts for the first $n$ games: $x_1, \ldots, x_n$. (Hint: remember that the log of the likelihood has the same maximizer as the likelihood function itself.)

$$L(\lambda) = \prod_{i=1}^{n} \text{Poisson}(x_i|\lambda) = \prod_{i=1}^{n} e^{-\lambda}\frac{\lambda_i^x}{x_i!}$$

$$\log(L(\lambda)) = \log(\prod_{i=1}^{n} \text{Poisson}(x_i|\lambda)) = \sum_{i=1}^{n} \log(e^{-\lambda}\frac{\lambda_i^x}{x_i!})$$

$$\log(L(\lambda)) = \sum_{i=1}^{n}(\log(e^{-\lambda}) + \log(\lambda^{x_i}) - \log(x_i!)) = \sum_{i=1}^{n}(-\lambda + x_i\log(\lambda) - \log(x_i!))$$

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}\Big(\log(L(\lambda))\Big) = 0 = \sum_{i=1}^{n}(-1 + \frac{x_i}{\lambda})$$

Thus we have

$$\sum_{i=1}^{n} 1 = \sum_{i=1}^{n} \frac{x_i}{\lambda}$$

$$\lambda = \frac{1}{n}\sum_{i=1}^{n} x_i$$

b. *[2 points]* Give a numerical estimate of $\lambda$ after the first five games. Given this $\lambda$, what is the probability that the Reign score exactly 6 goals in their next game?

$$\lambda = \sum_{i=1}^{5} \frac{x_i}{5} = \frac{2+4+6+0+1}{5} = \frac{13}{5}$$

$$\text{Poi}(6|\frac{13}{5}) = e^{-\frac{13}{5}}\left(\frac{\frac{13}{5}^6}{6!}\right) \approx 0.032 = 3.2\%$$

c. *[2 points]* Suppose the Reign score 8 goals in their 6th game. Give an updated numerical estimate of $\lambda$ after six games and compute the probability that the Reign score exactly 6 goals in their 7th game.

$$\lambda = \sum_{i=1}^{6} \frac{x_i}{6} = \frac{2+4+6+0+1+8}{6} = \frac{21}{6}$$

$$\text{Poi}(6|\frac{21}{6}) = e^{-\frac{21}{6}}\left(\frac{\frac{21}{6}^6}{6!}\right) \approx 0.077 = 7.7\%$$

# Overfitting

B1. Suppose we have $N$ labeled samples $S = \{(x_i, y_i)\}_{i=1}^{N}$ drawn i.i.d. from an underlying distribution $\mathcal{D}$. Suppose we decide to break this set into a set $S_{\text{train}}$ of size $N_{\text{train}}$ and a set $S_{\text{test}}$ of size $N_{\text{test}}$ samples for our training and test set, so $N = N_{\text{train}} + N_{\text{test}}$, and $S = S_{\text{train}} \cup S_{\text{test}}$. Recall the definition of the true least squares error of $f$:

$$\epsilon(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(f(x) - y)^2],$$

where the subscript $(x, y) \sim \mathcal{D}$ makes clear that our input-output pairs are sampled according to $\mathcal{D}$. Our training and test losses are defined as:

$$\widehat{\epsilon}_{\text{train}}(f) = \frac{1}{N_{\text{train}}} \sum_{(x,y)\in S_{\text{train}}} (f(x) - y)^2$$

$$\widehat{\epsilon}_{\text{test}}(f) = \frac{1}{N_{\text{test}}} \sum_{(x,y)\in S_{\text{test}}} (f(x) - y)^2$$

We then train our algorithm using the training set to obtain $\widehat{f}$.

a. *[2 points]* (bias: the test error) For all fixed $f$ (before we've seen any data) show that

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)] = \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)] = \epsilon(f).$$

Use a similar line of reasoning to show that the test error is an unbiased estimate of our true error for $\hat{f}$. Specifically, show that:

$$\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f})] = \epsilon(\widehat{f})$$

Starting with $\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]$, we show that

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)] = \mathbb{E}_{\text{train}}\left[\frac{1}{N_{\text{train}}} \sum_{(x,y)\in S_{\text{train}}} (f(x) - y)^2\right]$$

$$= \left(\frac{1}{N_{\text{train}}}\right) \mathbb{E}_{\text{train}}\left[\sum_{(x,y)\in S_{\text{train}}} (f(x) - y)^2\right]$$

$f$ is fixed and not related to the $S_{\text{train}} \iff f$ is independent. Since the input-output pairs in $S_{\text{train}}$ are *i.i.d* and $f$ is independent, sampling the entire set has the same expected value as sampling a point $N_{\text{train}}$ times from an underlying distribution $\mathcal{D}$. Thus

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)] = \left(\frac{1}{N_{\text{train}}}\right)(N_{\text{train}})\,\mathbb{E}_{(x,y)\sim\mathcal{D}}[(f(x) - y)^2] = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(f(x) - y)^2] = \epsilon(f)$$

Similarly for $\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]$, we have that

$$\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)] = \mathbb{E}_{\text{test}}\left[\frac{1}{N_{\text{test}}} \sum_{(x,y)\in S_{\text{test}}} (f(x) - y)^2\right]$$

$$= \left(\frac{1}{N_{\text{test}}}\right) \mathbb{E}_{\text{test}}\left[\sum_{(x,y)\in S_{\text{test}}} (f(x) - y)^2\right]$$

$f$ is fixed and not related to the $S_{\text{test}} \iff f$ is independent. Since the input-output pairs in $S_{\text{test}}$ are $i.i.d$ and $f$ is independent, sampling the entire set has the same expected value as sampling a point $N_{\text{test}}$ times from an underlying distribution $\mathcal{D}$. Thus

$$\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)] = \left(\frac{1}{N_{\text{test}}}\right)(N_{\text{test}})\,\mathbb{E}_{(x,y)\sim\mathcal{D}}[(f(x)-y)^2] = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(f(x)-y)^2] = \epsilon(f)$$

Therefore

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)] = \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)] = \epsilon(f).$$

We can make a similar proof to show that test error is an unbiased estimate of our true error for $\hat{f}$, i.e., $\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f})] = \epsilon(\widehat{f})$

$$\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f})] = \mathbb{E}_{\text{test}}\left[\frac{1}{N_{\text{test}}}\sum_{(x,y)\in S_{\text{test}}}(\widehat{f}(x)-y)^2\right]$$

$$= \left(\frac{1}{N_{\text{test}}}\right)\mathbb{E}_{\text{test}}\left[\sum_{(x,y)\in S_{\text{test}}}(\widehat{f}(x)-y)^2\right]$$

$\hat{f}$ is fixed and not related to the $S_{\text{test}} \iff \hat{f}$ is independent. Since the input-output pairs in $S_{\text{test}}$ are $i.i.d$ and $\hat{f}$ is independent, sampling the entire set has the same expected value as sampling a point $N_{\text{test}}$ times from an underlying distribution $\mathcal{D}$. Thus

$$\mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f})] = \left(\frac{1}{N_{\text{test}}}\right)(N_{\text{test}})\,\mathbb{E}_{(x,y)\sim\mathcal{D}}[(\widehat{f}(x)-y)^2] = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(\widehat{f}(x)-y)^2] = \epsilon(\widehat{f})$$

b. *[3 points]* (bias: the train/dev error) Is the above equation true (in general) with regards to the training loss? Specifically, does $\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f})]$ equal $\epsilon(\widehat{f})$? If so, why? If not, give a clear argument as to where your previous argument breaks down.

No, the above equation is not generally true with regard to the training loss. The equality

$$\left(\frac{1}{N_{\text{train}}}\right)\mathbb{E}_{\text{train}}\left[\sum_{(x,y)\in S_{\text{train}}}(\widehat{f}(x)-y)^2\right] = \left(\frac{1}{N_{\text{train}}}\right)(N_{\text{train}})\,\mathbb{E}_{(x,y)\sim\mathcal{D}}[(\widehat{f}(x)-y)^2]$$

would not hold because $\widehat{f}$ is not independent of $S_{\text{train}}$ and thus we cannot assume that $(\widehat{f}(x)-y)^2$ is $i.i.d$ in $\sum_{(x,y)\in S_{\text{train}}}(\widehat{f}(x)-y)^2)$.

c. *[5 points]* Let $\mathcal{F} = (f_1, f_2, \dots)$ be a collection of functions and let $\widehat{f}_{\text{train}}$ minimize the training error such that $\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}}) \leq \widehat{\epsilon}_{\text{train}}(f)$ for all $f \in \mathcal{F}$. Show that

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}})] \leq \mathbb{E}_{\text{train,test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f}_{\text{train}})].$$

(Hint: note that

$$\begin{aligned}\mathbb{E}_{\text{train,test}}[\widehat{\epsilon}_{\text{test}}(\widehat{f}_{\text{train}})] &= \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{train,test}}[\widehat{\epsilon}_{\text{test}}(f)\mathbf{1}\{\widehat{f}_{\text{train}} = f\}] \\ &= \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]\mathbb{E}_{\text{train}}[\mathbf{1}\{\widehat{f}_{\text{train}} = f\}] \\ &= \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]\mathbb{P}_{\text{train}}(\widehat{f}_{\text{train}} = f)\end{aligned}$$

where the second equality follows from the independence between the train and test set.)

Starting from the definition of $\mathcal{F}$

$$\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}}) \leq \widehat{\epsilon}_{\text{train}}(f) \qquad \forall f \in \mathcal{F}$$

$$\mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}})] \leq \mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)]$$

The expected value of the error on the trained model evaluated by iterating over all the test samples is less than the expected value of the error when we train the model with the train set and use the test set to evaluate the error.

$$\widehat{f} = \min_{f\in\mathcal{F}} \widehat{\epsilon}_{\text{train}}(f)$$

Since $\widehat{f}_{\text{train}}$ minimizes the training error, by the law of total expectation

$$\mathbb{E}_{\text{train,train}}[\widehat{\epsilon}_{\text{test}}(\widehat{f}_{\text{train}})] = \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{train,test}}[\widehat{\epsilon}_{\text{test}}(f)\mathbf{1}\{\widehat{f}_{\text{train}} = f\}]$$

$$= \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]\mathbb{E}_{\text{train}}[\mathbf{1}\{\widehat{f}_{\text{train}} = f\}] = \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{test}}[\widehat{\epsilon}_{\text{test}}(f)]\mathbb{P}_{\text{train}}[\widehat{f}_{\text{train}} = f]$$

$$= \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(f)]\mathbb{P}_{\text{train}}[\widehat{f}_{\text{train}} = f] \geq \sum_{f\in\mathcal{F}} \mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}})]\mathbb{P}_{\text{train}}[\widehat{f}_{\text{train}} = f]$$

$$= \mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}})] \sum_{f\in\mathcal{F}} \mathbb{P}_{\text{train}}[\widehat{f}_{\text{train}} = f] = \mathbb{E}_{\text{train}}[\widehat{\epsilon}_{\text{train}}(\widehat{f}_{\text{train}})]$$

## Bias-Variance tradeoff

B2. For $i = 1, \ldots, n$ let $x_i = i/n$ and $y_i = f(x_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ for some unknown $f$ we wish to approximate at values $\{x_i\}_{i=1}^n$. We will approximate $f$ with a step function estimator. For some $m \leq n$ such that $n/m$ is an integer define the estimator

$$\widehat{f}_m(x) = \sum_{j=1}^{n/m} c_j \mathbf{1}\{x \in \left(\frac{(j-1)m}{n}, \frac{jm}{n}\right]\} \quad \text{where} \quad c_j = \frac{1}{m} \sum_{i=(j-1)m+1}^{jm} y_i.$$

Note that $x \in \left(\frac{(j-1)m}{n}, \frac{jm}{n}\right]$ means $x$ is in the open-closed interval $\left(\frac{(j-1)m}{n}, \frac{jm}{n}\right]$.

Note that this estimator just partitions $\{1, \ldots, n\}$ into intervals $\{1, \ldots, m\}, \{m+1, \ldots, 2m\}, \ldots, \{n-m+1, \ldots, n\}$ and predicts the average of the observations within each interval.
By the bias-variance decomposition at some $x_i$ we have

$$\mathbb{E}\left[(\widehat{f}_m(x_i) - f(x_i))^2\right] = \underbrace{(\mathbb{E}[\widehat{f}_m(x_i)] - f(x_i))^2}_{\text{Bias}^2(x_i)} + \underbrace{\mathbb{E}\left[(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2\right]}_{\text{Variance}(x_i)}$$

a. *[5 points]* Intuitively, how do you expect the bias and variance to behave for small values of $m$? What about large values of $m$?

For small values of $m$, such as $m = 1$, I would expect the model to have low bias and high variance. For large values of $m$, such as $m = n$, I would expect the model to have high bias and low variance. Therefore, $m$ can be interpreted as inversely proportional to the complexity of the model. This is the case because as $m$ increases, there are fewer steps in the step function and thus a worse approximation.

b. *[5 points]* If we define $\bar{f}^{(j)} = \frac{1}{m}\sum_{i=(j-1)m+1}^{jm} f(x_i)$ and the *average bias-squared* as

$$\frac{1}{n}\sum_{i=1}^{n}(\mathbb{E}[\widehat{f}_m(x_i)] - f(x_i))^2 \ ,$$

show that

$$\frac{1}{n}\sum_{i=1}^{n}(\mathbb{E}[\widehat{f}_m(x_i)] - f(x_i))^2 = \frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}(\bar{f}^{(j)} - f(x_i))^2$$

We begin by considering $\mathbb{E}[\widehat{f}_m(x_i)]$, where $\frac{n}{m} \in \mathbb{Z}$

$$\mathbb{E}[\widehat{f}_m(x_i)] = \mathbb{E}[c_j] = \mathbb{E}[\frac{1}{m}\sum_{k=(j-1)m+1}^{jm} y_k] = \frac{1}{m}\sum_{k=(j-1)m+1}^{jm}\mathbb{E}[y_k] = \frac{1}{m}\sum_{k=(j-1)m+1}^{jm} f(x_k) = \bar{f}^{(j)}$$

Now we are able to show for the *average bias-squared* that

$$\frac{1}{n}\sum_{i=1}^{n}(\mathbb{E}[\widehat{f}_m(x_i)] - f(x_i))^2 = \frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}(\mathbb{E}[\widehat{f}_m(x_i)] - f(x_i))^2 = \frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}(\bar{f}^{(j)} - f(x_i))^2$$

c. *[5 points]* If we define the *average variance* as $\mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2\right]$, show (both equalities)

$$\mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2\right] = \frac{1}{n}\sum_{j=1}^{n/m} m\mathbb{E}[(c_j - \bar{f}^{(j)})^2] = \frac{\sigma^2}{m}$$

Since each $y_i$ is independent, we have that

$$\mathbb{E}[c_j] = \mathbb{E}\left[\frac{1}{m}\sum_{k=(j-1)m+1}^{jm} y_k\right] = \frac{1}{m^2}\sum_{k=(j-1)m+1}^{jm}\mathbb{E}[y_k] = \frac{\sigma^2}{m}$$

For each sample $x_i$ in a $j$th interval, the values of all the indicators of $\widehat{f}_m$ are 0 except those which correspond to the $j$th interval, thus we have $\widehat{f}_m(x_i) = c_j$.

$$\mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2\right] = \frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}\mathbb{E}[(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2]$$

$$= \frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}\mathbb{E}[(c_j - \mathbb{E}[c_j])^2] = \frac{1}{n}\sum_{j=1}^{n/m}(m)\mathbb{E}[(c_j - \bar{f}^{(j)})^2] = \frac{1}{n}\frac{n}{m}(m)\mathbb{E}[c_j] = \mathbb{E}[c_j]$$

Substituting for $\mathbb{E}[c_j]$, we get

$$\mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}(\widehat{f}_m(x_i) - \mathbb{E}[\widehat{f}_m(x_i)])^2\right] = \frac{1}{n}\sum_{j=1}^{n/m} m\mathbb{E}[(c_j - \bar{f}^{(j)})^2] = \frac{\sigma^2}{m}$$

d. *[5 points]* By the Mean-Value theorem we have that

$$\min_{i=(j-1)m+1,\ldots,jm} f(x_i) \leq \bar{f}^{(j)} \leq \max_{i=(j-1)m+1,\ldots,jm} f(x_i)$$

Suppose $f$ is $L$-Lipschitz so that $|f(x_i) - f(x_j)| \leq \frac{L}{n}|i-j|$ for all $i, j \in \{1, \ldots, n\}$ for some $L > 0$.

Show that the average bias-squared is $O(\frac{L^2m^2}{n^2})$. Using the expression for average variance above, the total error behaves like $O(\frac{L^2m^2}{n^2} + \frac{\sigma^2}{m})$. Minimize this expression with respect to $m$.

Does this value of $m$, and the total error when you plug this value of $m$ back in, behave in an intuitive way with respect to $n$, $L$, $\sigma^2$? That is, how does $m$ scale with each of these parameters? It turns out that this simple estimator (with the optimized choice of $m$) obtains the best achievable error rate up to a universal constant in this setup for this class of $L$-Lipschitz functions (see Tsybakov's *Introduction to Nonparametric Estimation* for details).

We previously derived the definition for *average bias-squared* as

$$\frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm}(\bar{f}^{(j)} - f(x_i))^2$$

To obtain bounds for the *average bias-squared*, we need to examine both the *min* and *max* cases.

$$(\bar{f}^{(j)} - f(x_i))^2 \leq (\min_{i=(j-1)m+1,\ldots,jm} f(x_i) - f(x_i))^2$$

$$(\bar{f}^{(j)} - f(x_i))^2 \leq (\max_{i=(j-1)m+1,\ldots,jm} f(x_i) - f(x_i))^2$$

According to the mean-value theorem, depending on $x_i$, one of the two cases is true for every term corresponding to a given interval $\sum_{i=(j-1)m+1}^{jm}(\bar{f}^{(j)} - f(x_i))^2$.

By the definition of the L-Lipschitz Continuity, we have

$$\min_{(j-1)m+1\leq i\leq jm} f(x_i) \leq \bar{f}^{(j)} \leq \max_{(j-1)m+1\leq i\leq jm} f(x_i) \quad \forall 1 \leq j \leq \frac{n}{m}$$

We are now able to establish bounds for both cases:

$$(\bar{f}^{(j)} - f(x_i))^2 \leq (\min_{i=(j-1)m+1,\ldots,jm} f(x_i) - f(x_i))^2 \leq \frac{L^2}{n^2}(\arg\min_{i=(j-1)m+1,\ldots,jm} f(x_i) - i)^2 \leq \frac{L^2}{n^2}(m-1)^2$$

$$(\bar{f}^{(j)} - f(x_i))^2 \leq (\max_{i=(j-1)m+1,\ldots,jm} f(x_i) - f(x_i))^2 \leq \frac{L^2}{n^2}(\arg\max_{i=(j-1)m+1,\ldots,jm} f(x_i) - i)^2 \leq \frac{L^2}{n^2}(m-1)^2$$

$$\sum_{i=(j-1)m+1}^{jm} (\bar{f}^{(j)} - f(x_i))^2 \leq m\frac{L^2}{n^2}(m-1)^2 \sim O(\frac{L^2m^3}{n^2})$$

Hence, we have

$$\frac{1}{n}\sum_{j=1}^{n/m}\sum_{i=(j-1)m+1}^{jm} (\bar{f}^{(j)} - f(x_i))^2 \sim O(\frac{1}{n}\frac{n}{m}\frac{L^2m^3}{n^2}) \sim O(\frac{L^2m^2}{n^2})$$

Using the expression for *average variance*, we see that the total error behaves like $O(\frac{L^2m^2}{n^2} + \frac{\sigma^2}{m})$. To solve for the value of $m$ that achieves minimal error, we take the derivative of this expression and set it equal to 0.

$$\frac{d}{dm}\left(\frac{L^2m^2}{n^2} + \frac{\sigma^2}{m}\right) = 0$$

$$2\frac{L^2m}{n^2} - \frac{\sigma^2}{m^2} = 0$$

$$m^3 = \frac{\sigma^2n^2}{2L^2}$$

$$m = \sqrt[3]{\frac{\sigma^2n^2}{2L^2}}$$

We take the second derivative with respect to $m$ to ensure that this is indeed a minima and not a maxima.

$$\frac{d}{dm}\left(2\frac{L^2m}{n^2} - \frac{\sigma^2}{m^2}\right) > 0$$

Since the second derivative is positive, the first derivative is always increasing, thus our expression for minimal error is in fact a minima. Plugging this newfound minimal error into $O(\frac{L^2m^2}{n^2})$ gives a total error that is $O\left(\frac{\sigma^{\frac{4}{3}}L^{\frac{2}{3}}}{n^{\frac{2}{3}}}\right)$.

Intuitively, increasing the amount of data, $n$, decreases the error and increases the allowable values of $m$ so that less complex models and be used. Decreasing $L$, making the underlying generating function more smooth and less jagged, decreases the error and also increases the allowable values of $m$. Increasing the amount of noise, $\sigma$, increases the error and increases $m$, making using simpler models more attractive in the presence of high noise to avoid fitting erroneous information.

## Polynomial Regression

- **polyreg.py**
- `linreg_closedform.py`
- `plot_polyreg_univariate.py`
- `plot_polyreg_learningCurve.py`

A3. Recall that polynomial regression learns a function $h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_d x^d$, where $d$ represents the polynomial's highest degree. We can equivalently write this in the form of a linear model with $d$ features

$$h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 \phi_1(x) + \theta_2 \phi_2(x) + \ldots + \theta_d \phi_d(x) \ , \tag{1}$$

using the basis expansion that $\phi_j(x) = x^j$. Notice that, with this basis expansion, we obtain a linear model where the features are various powers of the single univariate $x$. We're still solving a linear regression problem, but are fitting a polynomial function of the input.

a. *[8 points]* Implement regularized polynomial regression in `polyreg.py`. You may implement it however you like, using gradient descent or a closed-form solution.

```
class PolynomialRegression:
    @problem.tag("hw1-A", start_line=5)
    def __init__(self, degree: int = 1, reg_lambda: float = 1e-8):
        """Constructor
        """
        self.degree: int = degree
        self.reg_lambda: float = reg_lambda
        # Fill in with matrix with the correct shape
        self.weight: np.ndarray = None  # type: ignore
        # You can add additional fields

    @staticmethod
    @problem.tag("hw1-A")
    def polyfeatures(X: np.ndarray, degree: int) -> np.ndarray:
        """
        Expands the given X into an (n, degree) array of polynomial features of

        Args:
            X (np.ndarray): Array of shape (n, 1).
            degree (int): Positive integer defining maximum power to include.

        Returns:
            np.ndarray: A (n, degree) numpy array, with each row comprising of
                X, X * X, X ** 3, ... up to the degree^th power of X.
                Note that the returned matrix will not include the zero-th powe

        """
        powers = np.arange(1, degree + 1)
        return X**powers

    @problem.tag("hw1-A")
    def fit(self, X: np.ndarray, y: np.ndarray):
        """
        Trains the model, and saves learned weight in self.weight

        Args:
            X (np.ndarray): Array of shape (n, 1) with observations.
            y (np.ndarray): Array of shape (n, 1) with targets.
```

```
        Note:
            You will need to apply polynomial expansion and data standardizatior
        """
        train_size = len(X)
        X = self.polyfeatures(X, self.degree)
        self.mean = X.mean(axis=0)
        self.std = X.std(axis=0)
        X -= self.mean
        X /= self.std
        X = np.concatenate((np.ones([train_size, 1]), X), axis=1)
        n, d = X.shape

        reg_matrix = self.reg_lambda * np.eye(d)
        reg_matrix[0, 0] = 0
        self.weight = np.linalg.solve(X.T @ X + reg_matrix, X.T @ y)

    @problem.tag("hw1-A")
    def predict(self, X: np.ndarray) -> np.ndarray:
        """
        Use the trained model to predict values for each instance in X.

        Args:
            X (np.ndarray): Array of shape (n, 1) with observations.

        Returns:
            np.ndarray: Array of shape (n, 1) with predictions.
        """
        train_size = len(X)
        X = self.polyfeatures(X, self.degree)
        X -= self.mean
        X /= self.std
        X = np.concatenate((np.ones([train_size, 1]), X), axis=1)
        return X @ self.weight


@problem.tag("hw1-A")
def mean_squared_error(a: np.ndarray, b: np.ndarray) -> float:
    """Given two arrays: a and b, both of shape (n, 1) calculate a mean squared

    Args:
        a (np.ndarray): Array of shape (n, 1)
        b (np.ndarray): Array of shape (n, 1)

    Returns:
        float: mean squared error between a and b.
    """
    return (np.linalg.norm(a - b)**2)/len(a)


@problem.tag("hw1-A", start_line=5)
def learningCurve(
    Xtrain: np.ndarray,
    Ytrain: np.ndarray,
    Xtest: np.ndarray,
```

```
    Ytest: np.ndarray,
    reg_lambda: float,
    degree: int,
) -> Tuple[np.ndarray, np.ndarray]:
    """Compute learning curves.

    Args:
        Xtrain (np.ndarray): Training observations, shape: (n, 1)
        Ytrain (np.ndarray): Training targets, shape: (n, 1)
        Xtest (np.ndarray): Testing observations, shape: (n, 1)
        Ytest (np.ndarray): Testing targets, shape: (n, 1)
        reg_lambda (float): Regularization factor
        degree (int): Polynomial degree

    Returns:
        Tuple[np.ndarray, np.ndarray]: Tuple containing:
            1. errorTrain -- errorTrain[i] is the training mean squared error u
            2. errorTest -- errorTest[i] is the testing mean squared error using

    Note:
        - For errorTrain[i] only calculate error on Xtrain[0:(i+1)], since this
          THIS DOES NOT APPLY TO errorTest.
        - errorTrain[0:1] and errorTest[0:1] won't actually matter, since we st
    """
    n = len(Xtrain)

    errorTrain = np.zeros(n)
    errorTest = np.zeros(n)
    # Fill in errorTrain and errorTest arrays
    model = PolynomialRegression(degree, reg_lambda)
    for j in range(1, n):
        model.fit(Xtrain[:j + 1], Ytrain[:j + 1])
        predTrain = model.predict(Xtrain[:j + 1])
        predTest = model.predict(Xtest)
        errorTrain[j] = mean_squared_error(predTrain, Ytrain[:j + 1])
        errorTest[j] = mean_squared_error(predTest, Ytest)
    return errorTrain, errorTest
```

b. *[2 points]* Run `plot_polyreg_univariate.py` to test your implementation, which will plot the learned function. In this case, the script fits a polynomial of degree $d = 8$ with no regularization $\lambda = 0$. From the plot, we see that the function fits the data well, but will not generalize well to new data points. Try increasing the amount of regularization, and in 1-2 sentences, describe the resulting effect on the function (you may also provide an additional plot to support your analysis).

As we increase the amount of regularization, $\lambda$, the polynomial regression model's bias increases and variance decreases making it less prone to overfitting the data it observes. This allows the model to generalize better to unseen data points. When there is too much regularization, the model's bias becomes high making it prone to underfitting.

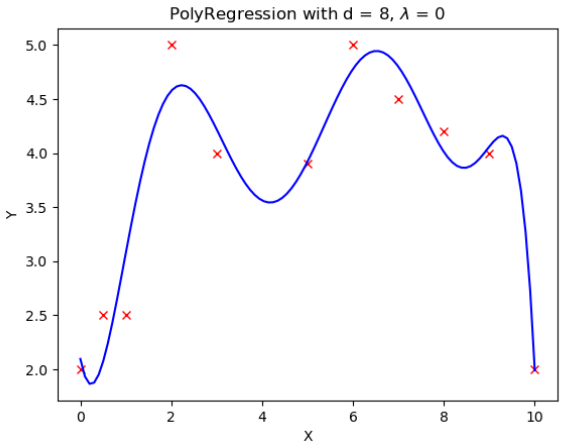



Figure 1: Fit of polynomial regression with degree $= 8$ and $\lambda = 0$

Now we apply varying amounts of regularization, with $\lambda = 0.01, 1$.

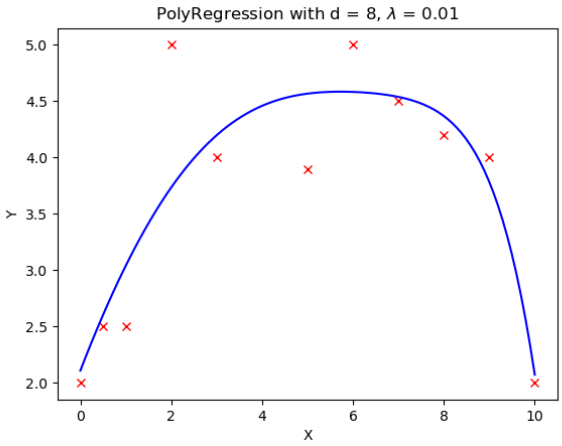



Figure 2: Fit of polynomial regression with degree $= 8$ and $\lambda = 0.01$

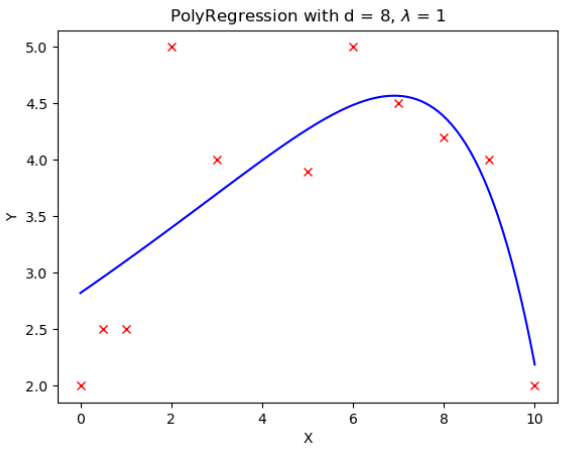



Figure 3: Fit of polynomial regression with degree = 8 and $\lambda = 1$

A4. *[10 points]* In this problem we will examine the bias-variance tradeoff through learning curves. Learning curves provide a valuable mechanism for evaluating the bias-variance tradeoff.
Implement the `learningCurve()` function in `polyreg.py` to compute the learning curves for a given training/test set. The `learningCurve(Xtrain, ytrain, Xtest, ytest, degree, regLambda)` function should take in the training data (`Xtrain`, `ytrain`), the testing data (`Xtest`, `ytest`), and values for the polynomial degree $d$ and regularization parameter $\lambda$. The function should return two arrays, `errorTrain` (the array of training errors) and `errorTest` (the array of testing errors). The $i^{th}$ index (start from 0) of each array should return the training error (or testing error) for learning with $i + 1$ training instances. Note that the $0^{th}$ index actually won't matter, since we typically start displaying the learning curves with two or more instances.

When computing the learning curves, you should learn on `Xtrain`$[0{:}i]$ for $i = 1, \ldots,$ numInstances(`Xtrain`), each time computing the testing error over the **entire** test set. There is no need to shuffle the training data, or to average the error over multiple trials – just produce the learning curves for the given training/testing sets with the instances in their given order. Recall that the error for regression problems is given by

$$\frac{1}{n}\sum_{i=1}^{n}(h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i)^2 \quad . \tag{2}$$

Once the function is written to compute the learning curves, run the `plot_polyreg_learningCurve.py` script to plot the learning curves for various values of $\lambda$ and $d$.
Notice the following:

- The y-axis is using a log-scale and the ranges of the y-scale are all different for the plots. The dashed black line indicates the $y = 1$ line as a point of reference between the plots.
- The plot of the unregularized model with $d = 1$ shows poor training error, indicating a high bias (i.e., it is a standard univariate linear regression fit).
- The plot of the (almost) unregularized model ($\lambda = 10^{-6}$) with $d = 8$ shows that the training error is low, but that the testing error is high. There is a huge gap between the training and testing errors caused by the model overfitting the training data, indicating a high variance problem.
- As the regularization parameter increases (e.g., $\lambda = 1$) with $d = 8$, we see that the gap between the training and testing error narrows, with both the training and testing errors converging to a low value. We can see that the model fits the data well and generalizes well, and therefore does not have either a high bias or a high variance problem. Effectively, it has a good tradeoff between bias and variance.
- Once the regularization parameter is too high ($\lambda = 100$), we see that the training and testing errors are once again high, indicating a poor fit. Effectively, there is too much regularization, resulting in high bias.

Submit plots for the same values of $d$ and $\lambda$ shown here. Make absolutely certain that you understand these observations, and how they relate to the learning curve plots. In practice, we can choose the value for $\lambda$ via cross-validation to achieve the best bias-variance tradeoff.

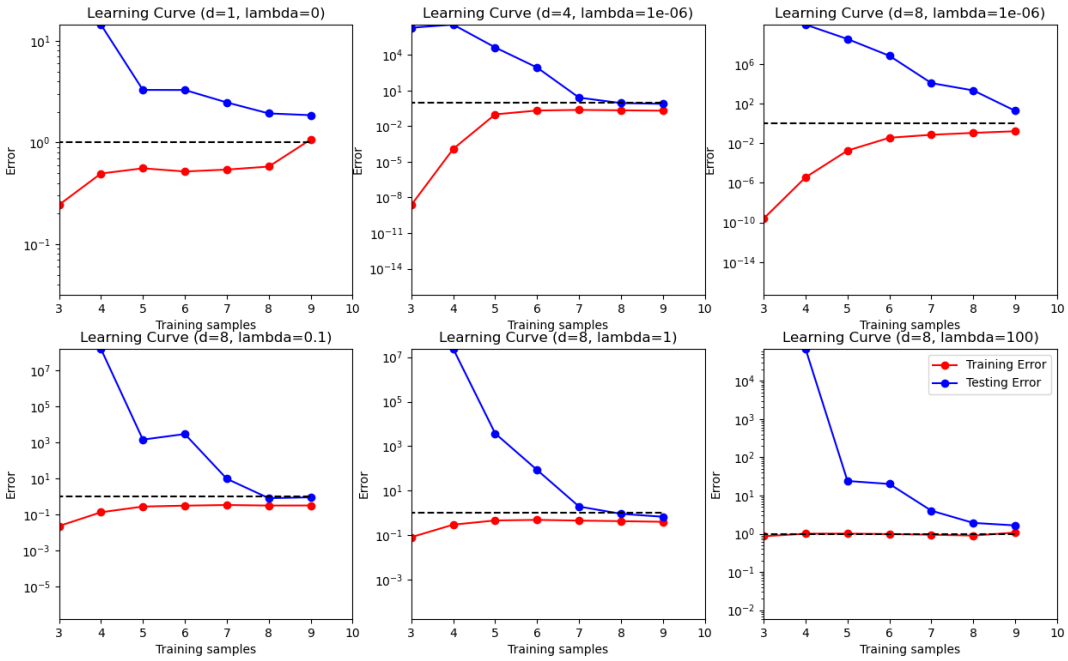



Figure 4: Plots of learning curves for $(d, \lambda) \in \{(1, 0), (4, 10^{-6}), (8, 10^{-6}), (8, 0.1), (8, 1), (8, 100)\}$

```
class PolynomialRegression:
    @problem.tag("hw1-A", start_line=5)
    def __init__(self, degree: int = 1, reg_lambda: float = 1e-8):
        """Constructor
        """
        self.degree: int = degree
        self.reg_lambda: float = reg_lambda
        # Fill in with matrix with the correct shape
        self.weight: np.ndarray = None  # type: ignore
        # You can add additional fields

    @staticmethod
    @problem.tag("hw1-A")
    def polyfeatures(X: np.ndarray, degree: int) -> np.ndarray:
        """
        Expands the given X into an (n, degree) array of polynomial features of degree d

        Args:
            X (np.ndarray): Array of shape (n, 1).
            degree (int): Positive integer defining maximum power to include.

        Returns:
            np.ndarray: A (n, degree) numpy array, with each row comprising of
```

```
                X, X * X, X ** 3, ... up to the degree^th power of X.
                Note that the returned matrix will not include the zero-th power.

        """
        powers = np.arange(1, degree + 1)
        return X**powers

    @problem.tag("hw1-A")
    def fit(self, X: np.ndarray, y: np.ndarray):
        """
        Trains the model, and saves learned weight in self.weight

        Args:
            X (np.ndarray): Array of shape (n, 1) with observations.
            y (np.ndarray): Array of shape (n, 1) with targets.

        Note:
            You will need to apply polynomial expansion and data standardization first.
        """
        train_size = len(X)
        X = self.polyfeatures(X, self.degree)
        self.mean = X.mean(axis=0)
        self.std = X.std(axis=0)
        X -= self.mean
        X /= self.std
        X = np.concatenate((np.ones([train_size, 1]), X), axis=1)
        n, d = X.shape

        reg_matrix = self.reg_lambda * np.eye(d)
        reg_matrix[0, 0] = 0
        self.weight = np.linalg.solve(X.T @ X + reg_matrix, X.T @ y)

    @problem.tag("hw1-A")
    def predict(self, X: np.ndarray) -> np.ndarray:
        """
        Use the trained model to predict values for each instance in X.

        Args:
            X (np.ndarray): Array of shape (n, 1) with observations.

        Returns:
            np.ndarray: Array of shape (n, 1) with predictions.
        """
        train_size = len(X)
        X = self.polyfeatures(X, self.degree)
        X -= self.mean
        X /= self.std
        X = np.concatenate((np.ones([train_size, 1]), X), axis=1)
        return X @ self.weight


@problem.tag("hw1-A")
def mean_squared_error(a: np.ndarray, b: np.ndarray) -> float:
    """Given two arrays: a and b, both of shape (n, 1) calculate a mean squared error.
```

```
    Args:
        a (np.ndarray): Array of shape (n, 1)
        b (np.ndarray): Array of shape (n, 1)

    Returns:
        float: mean squared error between a and b.
    """
    return (np.linalg.norm(a - b)**2)/len(a)


@problem.tag("hw1-A", start_line=5)
def learningCurve(
    Xtrain: np.ndarray,
    Ytrain: np.ndarray,
    Xtest: np.ndarray,
    Ytest: np.ndarray,
    reg_lambda: float,
    degree: int,
) -> Tuple[np.ndarray, np.ndarray]:
    """Compute learning curves.

    Args:
        Xtrain (np.ndarray): Training observations, shape: (n, 1)
        Ytrain (np.ndarray): Training targets, shape: (n, 1)
        Xtest (np.ndarray): Testing observations, shape: (n, 1)
        Ytest (np.ndarray): Testing targets, shape: (n, 1)
        reg_lambda (float): Regularization factor
        degree (int): Polynomial degree

    Returns:
        Tuple[np.ndarray, np.ndarray]: Tuple containing:
            1. errorTrain -- errorTrain[i] is the training mean squared error using mod
            2. errorTest -- errorTest[i] is the testing mean squared error using model

    Note:
        - For errorTrain[i] only calculate error on Xtrain[0:(i+1)], since this is the
          THIS DOES NOT APPLY TO errorTest.
        - errorTrain[0:1] and errorTest[0:1] won't actually matter, since we start displ
    """
    n = len(Xtrain)

    errorTrain = np.zeros(n)
    errorTest = np.zeros(n)
    # Fill in errorTrain and errorTest arrays
    model = PolynomialRegression(degree, reg_lambda)
    for j in range(1, n):
        model.fit(Xtrain[:j + 1], Ytrain[:j + 1])
        predTrain = model.predict(Xtrain[:j + 1])
        predTest = model.predict(Xtest)
        errorTrain[j] = mean_squared_error(predTrain, Ytrain[:j + 1])
        errorTest[j] = mean_squared_error(predTest, Ytest)
    return errorTrain, errorTest
```

# Ridge Regression on MNIST

**Relevant Files** (you should not need to modify any of the other files for this part):

- **ridge_regression.py**

A5. In this problem, we will implement a regularized least squares classifier for the MNIST data set. The task is to classify handwritten images of numbers between 0 to 9.

You are **NOT** allowed to use any of the pre-built classifiers in `sklearn`. Feel free to use any method from `numpy` or `scipy`. **Remember:** if you are inverting a matrix in your code, you are probably doing something wrong (Hint: look at `scipy.linalg.solve`).

Each example has features $x_i \in \mathbb{R}^d$ (with $d = 28 * 28 = 784$) and label $z_j \in \{0, \ldots, 9\}$. **Since images are represented as 784-dimensional vectors, you can visualize a single example $x_i$ with `imshow` after reshaping it to its original $28 \times 28$ image shape** (and noting that the label $z_j$ is accurate). We wish to learn a predictor $\widehat{f}$ that takes as input a vector in $\mathbb{R}^d$ and outputs an index in $\{0, \ldots, 9\}$. We define our training and testing classification error on a predictor $f$ as

$$\widehat{\epsilon}_{\text{train}}(f) = \frac{1}{N_{\text{train}}} \sum_{(x,z) \in \text{Training Set}} \mathbf{1}\{f(x) \neq z\}$$

$$\widehat{\epsilon}_{\text{test}}(f) = \frac{1}{N_{\text{test}}} \sum_{(x,z) \in \text{Test Set}} \mathbf{1}\{f(x) \neq z\}$$

We will use one-hot encoding of the labels: for each observation $(x, z)$, the original label $z \in \{0, \ldots, 9\}$ is mapped to the standard basis vector $e_{z+1}$ where $e_i$ is a vector of size $k$ containing all zeros except for a 1 in the $i^{\text{th}}$ position (positions in these vectors are indexed starting at one, hence the $z + 1$ offset for the digit labels). We adopt the notation where we have $n$ data points in our training objective with features $x_i \in \mathbb{R}^d$ and label one-hot encoded as $y_i \in \{0, 1\}^k$. Here, $k = 10$ since there are 10 digits.

a. *[10 points]* In this problem we will choose a linear classifier to minimize the regularized least squares objective:

$$\widehat{W} = \text{argmin}_{W \in \mathbb{R}^{d \times k}} \sum_{i=1}^{n} \|W^T x_i - y_i\|_2^2 + \lambda \|W\|_F^2$$

Note that $\|W\|_F$ corresponds to the Frobenius norm of $W$, i.e. $\|W\|_F^2 = \sum_{i=1}^{d} \sum_{j=1}^{k} W_{i,j}^2$. To classify a point $x_i$ we will use the rule $\arg\max_{j=0,\ldots,9} e_{j+1}^T \widehat{W}^T x_i$. Note that if $W = \begin{bmatrix} w_1 & \ldots & w_k \end{bmatrix}$ then

$$\begin{aligned} \sum_{i=1}^{n} \|W^T x_i - y_i\|_2^2 + \lambda \|W\|_F^2 &= \sum_{j=1}^{k} \left[ \sum_{i=1}^{n} (e_j^T W^T x_i - e_j^T y_i)^2 + \lambda \|W e_j\|^2 \right] \\ &= \sum_{j=1}^{k} \left[ \sum_{i=1}^{n} (w_j^T x_i - e_j^T y_i)^2 + \lambda \|w_j\|^2 \right] \\ &= \sum_{j=1}^{k} \left[ \|X w_j - Y e_j\|^2 + \lambda \|w_j\|^2 \right] \end{aligned}$$

where $X = \begin{bmatrix} x_1 & \ldots & x_n \end{bmatrix}^\top \in \mathbb{R}^{n \times d}$ and $Y = \begin{bmatrix} y_1 & \ldots & y_n \end{bmatrix}^\top \in \mathbb{R}^{n \times k}$. Show that

$$\widehat{W} = (X^T X + \lambda I)^{-1} X^T Y$$

We begin by representing $\widehat{W}$ column-wise, where $(X^TX+\lambda I)^{-1}X^TY_j$ is the $j$th column of $\widehat{W} \in \mathbb{R}^{d\times k}$. In order to minimize $\sum_{j=1}^{k}\left[\|Xw_j - Ye_j\|^2 + \lambda\|w_j\|^2\right]$, we take the partial derivative with respect to w and set it equal to zero.

$$\frac{\partial}{\partial w_j}\left(\sum_{j=1}^{k}\|Xw_j - Ye_j\|^2 + \lambda\|w_j\|^2\right) = 0$$

$$0 = \frac{\partial}{\partial w_j}\left[(Xw_j - Ye_j)^T(Xw_j - Ye_j) + \lambda w^T w\right]$$

$$0 = (2X^TXw_j - 2X^TYe_j + 2\lambda w_j) = 2(X^TXw_j - X^TYe_j + \lambda I w_j)$$

$$(X^TXw_j - X^TYe_j + \lambda w_j) = 0$$

$$(X^TXw_j + \lambda I w_j) = (X^TYe_j)$$

$$w_j(X^TX + \lambda I) = (X^TYe_j)$$

$$w_j = \frac{X^TYe_j}{X^TX + \lambda I}$$

$$w_j = (X^TX + \lambda I)^{-1}(X^TYe_j)$$

Since $e_j$ is a basis vector, we have that $Ye_j = Y_j$. Hence

$$w_j = (X^TX + \lambda I)^{-1}(X^TY_j)$$

Subsequently, since $w_j$ is the $j$th column of $\widehat{W}$, we have shown that

$$\widehat{W} = (X^TX + \lambda I)^{-1}X^TY$$

b. *[9 points]*

- Implement a function `train` that takes as input $X \in \mathbb{R}^{n\times d}$, $Y \in \{0,1\}^{n\times k}$, $\lambda > 0$ and returns $\widehat{W} \in \mathbb{R}^{d\times k}$.
- Implement a function `one_hot` that takes as input $Y \in \{0,...,k-1\}^n$, and returns $Y \in \{0,1\}^{n\times k}$.
- Implement a function `predict` that takes as input $W \in \mathbb{R}^{d\times k}$, $X' \in \mathbb{R}^{m\times d}$ and returns an $m$-length vector with the $i$th entry equal to $\arg\max_{j=0,...,9} e_j^T W^T x_i'$ where $x_i' \in \mathbb{R}^d$ is a column vector representing the $i$th example from $X'$.
- Using the functions you coded above, train a model to estimate $\widehat{W}$ on the MNIST training data with $\lambda = 10^{-4}$, and make label predictions on the test data. This behavior is implemented in the `main` function provided in a zip file.

```
@problem.tag("hw1-A")
def train(x: np.ndarray, y: np.ndarray, _lambda: float) -> np.ndarray:
    """Train function for the Ridge Regression problem.
    Should use observations ('x'), targets ('y') and regularization parameter (
    to train a weight matrix $$\\hat{W}$$.


    Args:
        x (np.ndarray): observations represented as '(n, d)' matrix.
            n is number of observations, d is number of features.
        y (np.ndarray): targets represented as '(n, k)' matrix.
            n is number of observations, k is number of classes.
        _lambda (float): parameter for ridge regularization.

    Raises:
        NotImplementedError: When problem is not attempted.
```

```
    Returns:
        np.ndarray: weight matrix of shape `(d, k)`
            which minimizes Regularized Squared Error on `x` and `y` with hyperp
    """
    a = x.T @ x + _lambda * np.eye(x.shape[1])
    b = x.T @ y
    return np.linalg.solve(a, b)


@problem.tag("hw1-A")
def predict(x: np.ndarray, w: np.ndarray) -> np.ndarray:
    """Train function for the Ridge Regression problem.
    Should use observations (`x`), and weight matrix (`w`) to generate predicate

    Args:
        x (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
        w (np.ndarray): weights represented as `(d, k)` matrix.
            d is number of features, k is number of classes.

    Raises:
        NotImplementedError: When problem is not attempted.

    Returns:
        np.ndarray: predictions matrix of shape `(n,)` or `(n, 1)`.
    """
    return np.argmax(x @ w, axis=1)


@problem.tag("hw1-A")
def one_hot(y: np.ndarray, num_classes: int) -> np.ndarray:
    """One hot encode a vector `y`.
    One hot encoding takes an array of integers and coverts them into binary for
    Each number i is converted into a vector of zeros (of size num_classes), wi

    Args:
        y (np.ndarray): An array of integers [0, num_classes), of shape (n,)
        num_classes (int): Number of classes in y.

    Returns:
        np.ndarray: Array of shape (n, num_classes).
        One-hot representation of y (see below for example).

    Example:
        ```python
        > one_hot([2, 3, 1, 0], 4)
        [
            [0, 0, 1, 0],
            [0, 0, 0, 1],
            [0, 1, 0, 0],
            [1, 0, 0, 0],
        ]
        ```
    """
```

```
    arr = np.zeros([len(y), num_classes])
    for i in range(len(y)):
        arr[i][y[i]] = 1
    return arr
```

c. *[1 point]* What are the training and testing errors of the classifier trained as above?
Train Error: 14.805%
Test Error: 14.66%

d. *[2 points]* Using matplotlib's `imshow` function, plot any 10 samples from the test data whose labels are incorrectly predicted by the classifier. Notice any patterns?

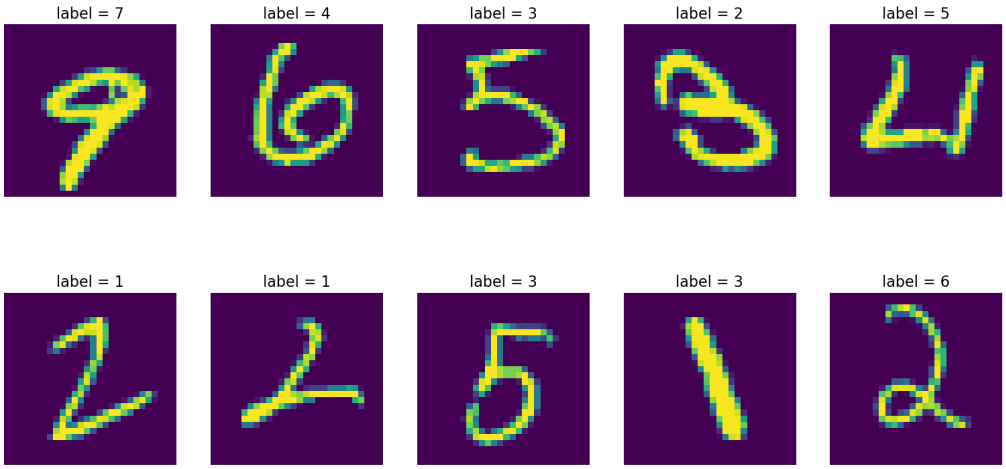



Figure 5: 10 MNIST samples incorrectly classified using ridge regression.

The reason these digits were incorrectly classified is likely that the model we trained was too simple to accurately capture all the relevant features, this could potentially be rectified with more training data. Additionally, some of the digits that were incorrectly classified were rather ambiguous.

Once you finish this problem question, you should have a powerful handwritten digit classifier! Curious to know how it compares to other models, including the almighty *Neural Networks*? Check out the **linear classifier (1-layer NN)** on the official MNIST leaderboard. (The model we just built is actually a 1-layer neural network: more on this soon!)

## Administrative

A6.

a. *[2 points]* About how many hours did you spend on this homework? There is no right or wrong answer :)

I spent about 20 hours on this homework.